

OBJECT ORIENTED PROGRAMMING WITH C++

DSA0103

CO2-AT2-SCENARIO BASED ASSESMENT

Name: Ranjitha .R

Reg no: 192421285

1. A hospital wants to estimate consultation charges for different patient categories. Design a C++ program for a Consultation Charge System. Design a C++ program for hospital management

Task:

- A. Use function overloading to calculate charge for general consultation and specialist consultation.
- b. Use call by reference to modify the final bill amount.
- c. Use default arguments to add a standard service charge if needed.
- d. Use function prototyping for all calculation functions.
- e. Display the patient type and total consultation charge.(BL3)

CODE:

```
#include <iostream>

using namespace std;

double generalConsultation(int patientType);

double specialistConsultation(int patientType);

double addServiceCharge(double bill, double serviceCharge = 50.0);

void updateFinalBill(double &finalBill, double subsidy);

int main() {

    int patientType;

    double finalBill = 0.0;

    double serviceCharge = 0.0;

    double subsidy = 0.0;

    cout << "Enter patient type (1-General, 2-Specialist): ";

    cin >> patientType;

    if (patientType == 1) {

        finalBill = generalConsultation(patientType);

    }

    else if (patientType == 2) {
```

```

        finalBill = specialistConsultation(patientType);
    }
    else {
        cout << "Invalid patient type." << endl;
        return 0;
    }
    finalBill = addServiceCharge(finalBill);
    cout << "Enter subsidy amount: ";
    cin >> subsidy;
    updateFinalBill(finalBill, subsidy);
    cout << "Final Bill: " << finalBill << endl;
    return 0;
}

double generalConsultation(int patientType) {
    return 500.0;
}

double specialistConsultation(int patientType) {
    return 1000.0;
}

double addServiceCharge(double bill, double serviceCharge) {
    return bill + serviceCharge;
}

void updateFinalBill(double &finalBill, double subsidy) {
    finalBill = finalBill - subsidy;
    if (finalBill < 0) {
        finalBill = 0;
    }
}
}

```

```
main.cpp  [Icons]  Share  Run  Output

41     return 0;
42 }
43
44 // General consultation
45 double generalConsultation(int patientType) {
46     return 500.0;
47 }
48
49 // Specialist consultation
50 double specialistConsultation(int patientType) {
51     return 1000.0;
52 }
53
54 // Add service charge
55 double addServiceCharge(double bill, double serviceCharge) {
56     return bill + serviceCharge;
57 }
58
59 // Update final bill using reference
60 void updateFinalBill(double &finalBill, double subsidy) {
61     finalBill = finalBill - subsidy;
62 }
63 if (finalBill < 0) {
64     finalBill = 0;
65 }
```

```
Enter patient type (1-General, 2-Specialist): 1
Enter subsidy amount: 100
100
Final Bill: 450

=== Code Execution Successful ===
```

2. Design a C++ program Fitness Centre Membership System. A gym wants to manage member subscriptions, track plans, and calculate membership fees.

Task:

Create a class Member with attributes: Member ID, Name, Plan Type, and Fee.

Implement member functions to assign plans, calculate fees, and display membership details.

Use an array of objects to manage multiple members.

Add validation for valid membership plans.

Include a static member to track total active members.

Evaluate how your design supports easy modification of pricing plans.

CODE:

```
#include <iostream>
#include <string>
using namespace std;
class Member {
private:
int memberID;
string name;
string planType;
double fee;
static int activeMembers; // static member
public:
Member() {
memberID = 0;
name = "";
planType = "";
fee = 0;
```

```

}
Member(int id, string n) {
    memberID = id;
    name = n;
    planType = "";
    fee = 0;
}
void assignPlan(string plan) {

    if (plan == "Monthly") fee = 500;
    else if (plan == "Quarterly") fee = 1300;
    else if (plan == "Yearly") fee = 4500;
    else {
        cout << "Invalid Plan! Choose Monthly/Quarterly/Yearly.\n";
        return;
    }

    planType = plan;
    activeMembers++;
}

double calculateFees() {
    return fee;
}

void displayDetails() {
    cout << "Member ID: " << memberID << "\n";
    cout << "Name: " << name << "\n";
    cout << "Plan Type: " << planType << "\n";
    cout << "Fee: " << fee << "\n\n";
}

static void displayTotalActiveMembers() {
    cout << "Total Active Members: " << activeMembers << "\n";
}
};

int Member::activeMembers = 0;

int main() {
    const int MAX = 3;
    Member members[MAX];

    for (int i = 0; i < MAX; i++) {
        int id;
        string name, plan;
        cout << "Enter Member " << i + 1 << " ID: ";
        cin >> id;
    }
}

```

```

cin.ignore();
cout << "Enter Name: ";
getline(cin, name);

members[i] = Member(id, name);

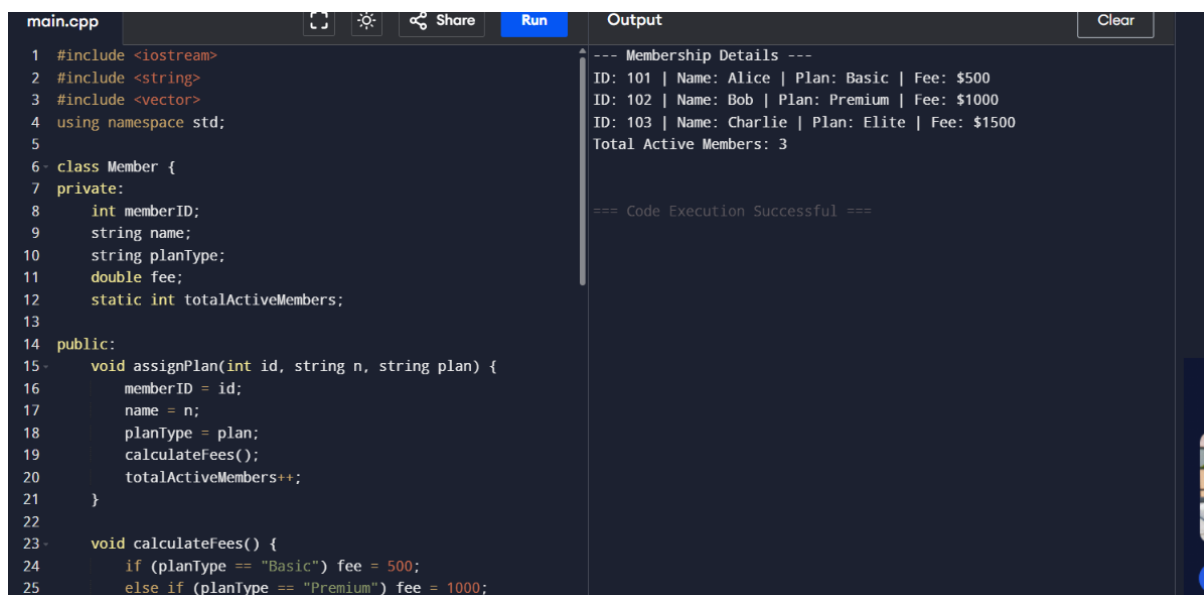
cout << "Enter Plan (Monthly/Quarterly/Yearly): ";
cin >> plan;

members[i].assignPlan(plan);
}

cout << "\n--- Membership Details ---\n";
for (int i = 0; i < MAX; i++) {
    members[i].displayDetails();
}

Member::displayTotalActiveMembers();
return 0;
}

```



The screenshot shows a C++ IDE with a dark theme. The left pane displays the source code for `main.cpp`, and the right pane shows the program's output.

Source Code (main.cpp):

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4 using namespace std;
5
6 class Member {
7 private:
8     int memberID;
9     string name;
10    string planType;
11    double fee;
12    static int totalActiveMembers;
13
14 public:
15    void assignPlan(int id, string n, string plan) {
16        memberID = id;
17        name = n;
18        planType = plan;
19        calculateFees();
20        totalActiveMembers++;
21    }
22
23    void calculateFees() {
24        if (planType == "Basic") fee = 500;
25        else if (planType == "Premium") fee = 1000;

```

Output:

```

--- Membership Details ---
ID: 101 | Name: Alice | Plan: Basic | Fee: $500
ID: 102 | Name: Bob | Plan: Premium | Fee: $1000
ID: 103 | Name: Charlie | Plan: Elite | Fee: $1500
Total Active Members: 3

=== Code Execution Successful ===

```

3. A bank wants to calculate loan repayment details for customers. Design a C++ program for a Loan Repayment System.

Task:

- A. Use function prototyping to declare EMI-related functions.
- b. Use default arguments for default interest rate where applicable.
- c. Apply call by reference to update payable amount after subsidy.
- d. Use inline functions for simple interest calculations.
- e. Display the loan amount, interest, and repayment details clearly.

CODE:

```
#include <iostream>
#include <cmath>
using namespace std;

// Function prototypes
double calculateEMI(double principal, double rate, int months);
double updatePayable(double payable, double& payableAfterSubsidy, double subsidy);

// Inline simple interest
inline double simpleInterest(double principal, double ratePerYear, int years) {
    return (principal * ratePerYear * years) / 100.0;
}

int main() {
    double principal, rate;
    int months;

    cout << "Enter Loan Amount (Principal): ";
    cin >> principal;

    cout << "Enter Interest Rate per month (%). (Enter 0 for default 1.5): ";
    cin >> rate;
    if (rate == 0) rate = 1.5;

    cout << "Enter Loan Duration (Months): ";
    cin >> months;

    double emi = calculateEMI(principal, rate, months);

    double payable = emi * months; // total payable
    double payableAfterSubsidy = payable;

    double subsidy;
    cout << "Enter Subsidy Amount: ";
    cin >> subsidy;

    updatePayable(payable, payableAfterSubsidy, subsidy);

    cout << "\n--- Loan Repayment Details ---\n";
    cout << "Loan Amount: " << principal << endl;
    cout << "Interest (monthly %): " << rate << endl;
    cout << "EMI: " << emi << endl;
    cout << "Total Payable (before subsidy): " << payable << endl;
    cout << "Payable after subsidy (call by reference updated): " << payableAfterSubsidy << endl;

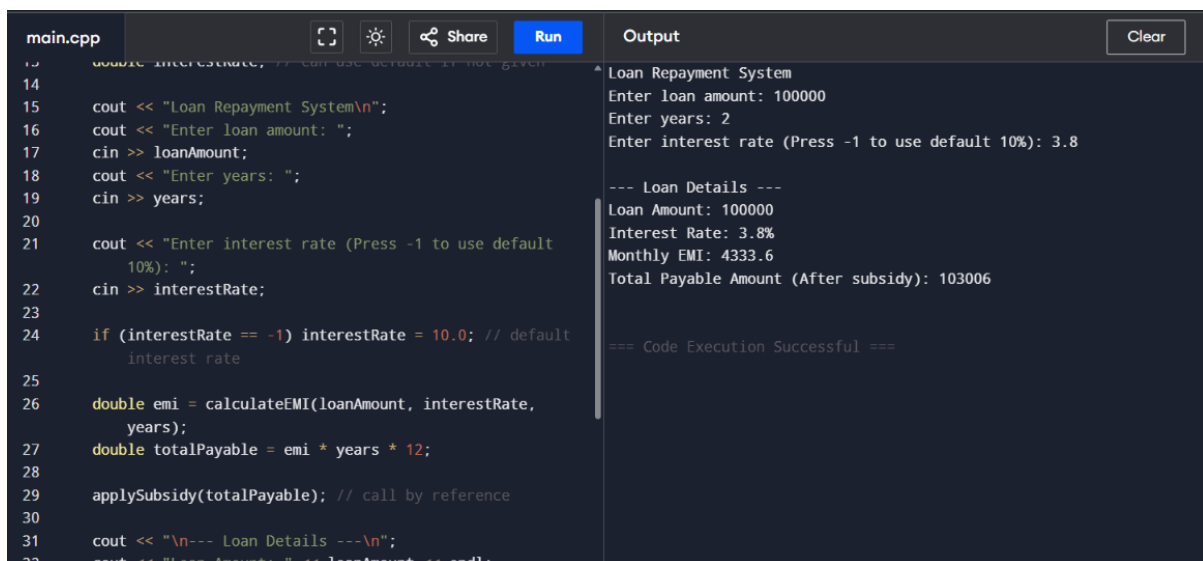
    return 0;
```

```
}
```

```
double calculateEMI(double principal, double rate, int months) {  
    double r = rate / 100.0;  
    double n = months;
```

```
    double emi = (principal * r * pow(1 + r, n)) / (pow(1 + r, n) - 1);  
    return emi;  
}
```

```
double updatePayable(double payable, double& payableAfterSubsidy, double subsidy) {  
    payableAfterSubsidy = payable - subsidy;  
    return payableAfterSubsidy;  
}
```



The screenshot shows a C++ IDE with a file named `main.cpp`. The code implements a loan repayment system. It prompts the user to enter the loan amount, years, and interest rate. If the interest rate is -1, it defaults to 10%. It then calculates the EMI using the `calculateEMI` function, calculates the total payable, and applies a subsidy using the `updatePayable` function. The output shows the loan details and the total payable amount after the subsidy.

```
13 // double interestRate; // can use default if not given  
14  
15 cout << "Loan Repayment System\n";  
16 cout << "Enter loan amount: ";  
17 cin >> loanAmount;  
18 cout << "Enter years: ";  
19 cin >> years;  
20  
21 cout << "Enter interest rate (Press -1 to use default  
    10%): ";  
22 cin >> interestRate;  
23  
24 if (interestRate == -1) interestRate = 10.0; // default  
    interest rate  
25  
26 double emi = calculateEMI(loanAmount, interestRate,  
    years);  
27 double totalPayable = emi * years * 12;  
28  
29 applySubsidy(totalPayable); // call by reference  
30  
31 cout << "\n--- Loan Details ---\n";  
32 cout << "Loan Amount: " << loanAmount << endl;
```

Output:

```
Loan Repayment System  
Enter loan amount: 100000  
Enter years: 2  
Enter interest rate (Press -1 to use default 10%): 3.8  
  
--- Loan Details ---  
Loan Amount: 100000  
Interest Rate: 3.8%  
Monthly EMI: 4333.6  
Total Payable Amount (After subsidy): 103006  
  
=== Code Execution Successful ===
```

4. A delivery company wants to automate the calculation of parcel delivery charges based on parcel weight, delivery distance, and delivery type. Design and implement a C++ program using functions and object-oriented programming concepts to develop a Parcel Charge Management System.

- Create a class `Parcel` with appropriate private data members such as parcel type, weight, distance, and packaging charge.
- Use public member functions to read parcel details, calculate the delivery charge, and display the parcel information.
- Use function overloading to calculate charges for Normal Delivery and Express Delivery.
- Use default arguments to assign a default packaging charge when it is not specified by the user.
- Use an inline function to calculate the base charge based on parcel weight and delivery distance.
- Maintain the total number of parcels processed using a static data member and display it through a static member function.

CODE:

```
#include <iostream>  
#include <string>  
using namespace std;  
  
class Parcel {
```

```

private:
string parcelType; // Normal / Express
double weight; // in kg
double distance; // in km
double packagingCharge;

static int totalParcels; // (f) maintain total number of parcels processed

inline double baseCharge(double w, double d) {
return (w * 10.0) + (d * 2.0);
}

public:

Parcel() : parcelType(""), weight(0), distance(0), packagingCharge(50.0) {}

void readDetails() {
cout << "Enter Parcel Type (Normal/Express): ";
cin >> parcelType;

cout << "Enter Weight (kg): ";
cin >> weight;

cout << "Enter Distance (km): ";
cin >> distance;

cout << "Enter Packaging Charge (or type -1 for default): ";
double temp;
cin >> temp;

if (temp == -1)
packagingCharge = 50.0; // default packaging charge
else
packagingCharge = temp;

totalParcels++;
}

double calculateCharge(double packCharge = 50.0) {
packagingCharge = packCharge;
return baseCharge(weight, distance) + packagingCharge;
}

double calculateCharge(double normalMultiplier, double expressMultiplier, double packCharge
= 50.0) {
packagingCharge = packCharge;

double base = baseCharge(weight, distance);

```



```
if (parcelType == "Normal" || parcelType == "normal") {
    return (base * normalMultiplier) + packagingCharge;
} else {
    return (base * expressMultiplier) + packagingCharge;
}
}
```

```
void displayInfo(double charge) {
    cout << "\n--- Parcel Information ---" << endl;
    cout << "Parcel Type: " << parcelType << endl;
    cout << "Weight (kg): " << weight << endl;
    cout << "Distance (km): " << distance << endl;
    cout << "Packaging Charge: " << packagingCharge << endl;
    cout << "Total Delivery Charge: " << charge << endl;
}
```

```
static void displayTotalParcels() {
    cout << "\nTotal Parcels Processed: " << totalParcels << endl;
}
};
```

```
int Parcel::totalParcels = 0;
```

```
int main() {
    int n;
    cout << "Enter number of parcels: ";
    cin >> n;
```

```
Parcel* parcels = new Parcel[n];
```

```
for (int i = 0; i < n; i++) {
    cout << "\nEnter details for Parcel " << (i + 1) << ": " << endl;
    parcels[i].readDetails();
}
```

```
double charge = parcels[i].calculateCharge(1.0, 1.5);
parcels[i].displayInfo(charge);
}
```

```
Parcel::displayTotalParcels();
delete[] parcels;
return 0;
}
```

main.cpp	Output
<pre> 87 cin >> type; 88 cout << "Enter weight (kg): "; 89 cin >> w; 90 cout << "Enter distance (km): "; 91 cin >> d; 92 93 cout << "Enter packaging charge (or -1 to use default 50): "; 94 cin >> pack; 95 96 if (pack == -1) 97 parcels[i].readDetails(type, w, d); // default packaging charge 98 else 99 parcels[i].readDetails(type, w, d, pack); 100 101 cout << "\nParcel Details:\n"; 102 parcels[i].displayParcelInfo(); 103 cout << "Calculated Charge:\n"; 104 parcels[i].calculateAndDisplayCharge(); 105 } 106 </pre>	<pre> Enter number of parcels: 3 --- Parcel 1 --- Enter type (Normal/Express): normal Enter weight (kg): 5 Enter distance (km): 5 Enter packaging charge (or -1 to use default 50): 34 Parcel Details: Weight: 5 kg Distance: 5 km Packaging Charge: 34 Calculated Charge: Invalid Parcel Type! --- Parcel 2 --- Enter type (Normal/Express): </pre>

5. A utility company wants to automate the calculation of water bills for domestic and commercial consumers. Design and implement a C++ program using functions and object-oriented programming concepts to develop a Water Billing System. create a class Consumer with appropriate private data members such as consumer ID, consumer type, water consumption (in liters), and bill amount.
- Use public member functions to input consumer details, calculate the water bill, and display the billing information.
 - Implement function overloading to calculate water bills for Domestic and Commercial consumers using different tariff rates.
 - Use call by reference to update the bill amount after applying a government subsidy.
 - Use default arguments to include a service tax, where a default tax rate is applied unless specified.
 - Declare all functions using function prototyping before their definitions.
 - Maintain the total number of consumers processed using a static data member and display it through a static member function.
 - Create an array of objects to store and process the details of multiple consumers, and display the consumer type, water consumption, subsidy-adjusted bill amount,

CODE:

```

#include <iostream>
#include <string>
using namespace std;

class Consumer {
private:
    int consumerID;
    string consumerType; // DOMESTIC/COMMERCIAL
    double liters;
    double billAmount;

    static int totalConsumers;

public:
    Consumer() {
        consumerID = 0;
        consumerType = "";
        liters = 0;
        billAmount = 0;
    }

```

```

void inputDetails(int id, string type, double l) {
    consumerID = id;
    consumerType = type;
    liters = l;
    totalConsumers++;
}

// Default service tax (default argument)
double calculateBill(double serviceTaxRate = 5.0) {
    if (consumerType == "DOMESTIC") {
        billAmount = calculateDomestic(liters, serviceTaxRate);
    } else {
        billAmount = calculateCommercial(liters, serviceTaxRate);
    }
    return billAmount;
}

// Overloading for Domestic and Commercial
double calculateDomestic(double l, double serviceTaxRate) {
    double tariff = 8.0; // rate per liter
    double base = l * tariff;
    double tax = base * (serviceTaxRate / 100.0);
    return base + tax;
}

double calculateCommercial(double l, double serviceTaxRate) {
    double tariff = 15.0; // higher rate
    double base = l * tariff;
    double tax = base * (serviceTaxRate / 100.0);
    return base + tax;
}

// call by reference to update bill after subsidy
void applySubsidy(double subsidy) {
    billAmount -= subsidy;
    if (billAmount < 0) billAmount = 0;
}

void display() const {
    cout << "Consumer ID: " << consumerID << endl;
    cout << "Consumer Type: " << consumerType << endl;
    cout << "Water Consumption: " << liters << " liters" << endl;
    cout << "Final Subsidy-Adjusted Bill: " << billAmount << endl;
    cout << "-----\n";
}

static void displayTotalConsumers() {
    cout << "Total Consumers Processed = " << totalConsumers << endl;
}

```

```
}  
};
```

```
int Consumer::totalConsumers = 0;
```

```
int main() {  
    int n;  
    cout << "Water Billing System\n";  
    cout << "Enter number of consumers: ";  
    cin >> n;
```

```
    Consumer consumers[50];
```

```
    for (int i = 0; i < n; i++) {  
        int id;  
        string type;  
        double liters, subsidy;  
  
        cout << "\nConsumer " << i + 1 << endl;  
        cout << "Enter Consumer ID: ";  
        cin >> id;  
        cout << "Enter Type (DOMESTIC/COMMERCIAL): ";  
        cin >> type;  
        cout << "Enter water consumption (liters): ";  
        cin >> liters;
```

```
        consumers[i].inputDetails(id, type, liters);
```

```
        double useTaxRate;  
        cout << "Enter service tax rate % (enter 0 to use default 5): ";  
        cin >> useTaxRate;
```

```
        if (useTaxRate == 0) consumers[i].calculateBill(); // default  
        else consumers[i].calculateBill(useTaxRate);
```

```
        cout << "Enter government subsidy (amount): ";  
        cin >> subsidy;
```

```
        consumers[i].applySubsidy(subsidy);  
        consumers[i].display();  
    }
```

```
    Consumer::displayTotalConsumers();  
    return 0;  
}
```

main.cpp	Run	Output
<pre>88 89 // default argument service tax 90 bill += bill * (serviceTaxRate / 100.0); 91 return bill; 92 } 93 94 int main() { 95 int n; 96 cout << "Enter number of consumers: "; 97 cin >> n; 98 99 Consumer consumers[20]; 100 101 double subsidyPercent; 102 cout << "Enter Subsidy Percent to apply (e.g., 15): "; 103 cin >> subsidyPercent; 104 105 for (int i = 0; i < n; i++) { 106 cout << "\nConsumer " << (i + 1) << ": \n"; 107 consumers[i].input(); 108 consumers[i].calculateAndDisplay(subsidyPercent); 109 } 110 111 Consumer::displayTotalConsumers(); 112 }</pre>	Run	<pre>Enter number of consumers: 4 Enter Subsidy Percent to apply (e.g., 15): 13 Consumer 1: Enter Consumer ID: 345 Enter Consumer Type (Domestic/Commercial): commercial Enter Water Consumption (liters): 5 Invalid consumer type. Consumer 2: Enter Consumer ID: </pre>